一、数据传送指令



- 数据传送指令是**最基本**、**最常用**的指令。
- 数据传送指令主要完成**两个部件之间的数据传送**操作。
  - **寄存器与寄存器**之间的数据传送
  - **寄存器与存储器**之间的数据传送
- 有的计算机采用**通用的MOV指令**实现数据传送操作。
- 有的计算机采用**LOAD指令**从主存单元**读取数据到寄存器**；采用**STORE指令**将寄存器中的**数据写入主存单元**。
- 数据传送指令可以**字节**、**字**、**双字**为单位进行数据传送。有的指令系统甚至还支持**成组数据**传送（例如Intel x86的指令系统）。

二、算数和逻辑运算指令



- 算术和逻辑运算指令用于**CPU所实现的最基本功能**。
  - 算术运算：加（ADD）、减（SUB）、乘（MUL）、除（DIV）等
  - 逻辑运算：与（AND）、或（OR）、非（NOT）、异或（XOR）等
- 不同架构的计算机对算术运算指令的支持有很大差别。
  - 有的计算机仅支持**二进制定点数的加、减、比较、求补**等最简单的指令，目的是为了让硬件实现最简单。
  - 有的计算机除了支持基本的算术运算指令外，还支持**浮点运算**指令，**十进制运算**指令，甚至支持**乘方**、**开方**、**多项式计算**等指令。
  - 有的大型机和巨型机除了支持**标量运算**指令外，还支持**向量运算**指令，可直接对整个**向量**或**矩阵**进行**求和**、**求积**运算。

三、移位操作指令



- 在本课程第3章的前三次课中，我们分别给同学们介绍了逻辑移位、算术移位，循环移位共三种移位操作。
- 移位操作指令就是用来告诉计算机要执行哪种移位操作，因此可分为：
  - 逻辑移位指令　移位对象是无符号数
  - 算术移位指令　移位对象是有符号数
  - 循环移位指令　移位对象是无符号数
    - 不带CF标志位的循环右移
    - 不带CF标志位的循环左移
    - 小循环
    - 带CF标志位的循环右移
    - 带CF标志位的循环左移
    - 大循环

四、程序控制指令

4.1 转移指令

转移指令　循环控制指令　子程序调用与返回指令

- 无条件转移指令
- 条件转移指令
  - 一般根据状态标志寄存器（或条件码寄存器）中的相关标志位，来决定是否进行转移操作。
  - 上述标志位一般包括进位标志CF、有符号溢出标志OF、结果为零标志ZF、结果为负标志NF等。
  - 上述标这些标志位一般由前面的指令根据执行结果来设置，并作为当前条件转移指令决定是否进行转移的条件。

4.2 循环控制指令

转移指令　　**循环控制指令**　　子程序调用与返回指令

- 为了实现循环程序的执行，循环控制指令一般包括以下功能：
  - 对循环控制变量的修改
  - 条件测试和判断
  - 地址转移
- 综上所述，循环控制指令实际上是功能更强的转移指令。

4.3 子程序调用与返回指令

转移指令　　循环控制指令　　**子程序调用与返回指令**

- 在主程序执行过程中，当需要调用公用的子程序时，可执行子程序调用指令，进入子程序执行。
- 当子程序执行完毕后，可以利用返回指令，从子程序返回到主程序继续执行。
- 子程序调用指令中必须明确给出子程序的入口地址。
- 在主程序中，子程序调用指令的下一条指令的地址称为断点，断点是子程序返回主程序时的返回地址。
- 为了使子程序执行返回指令时能够正确返回主程序，主程序中的子程序调用指令应当具备断点保护的功能，一般情况下，子程序调用指令会将断点（子程序的返回地址）进行压栈保护，而返回指令则从堆栈中取出断点并送入程序计数PC，这样就可使子程序返回到主程序中的断点处继续执行主程序。

4.4 转移指令、子程序调用与返回指令的对比

转移指令	VS	子程序调用与返回指令
● 条件转移指令需要满足条件		● 一般是无条件的
● 在同一程序内转移		● 不同程序之间转移
● 不需要返回原处		● 需要返回调用处，因此需要保护断点（返回地址）

五、输入输出指令

■ 输入输出指令一般简称为I/O指令，用于实现主机与外部设备之间的信息传送。

- □ 主机可向外部设备发出各种控制命令，从而控制外部设备的工作。
- □ 主机也可从外部设备端口寄存器中读取外部设备的各种工作状态。

■ 当外部设备与主存采用统一编址模式时，则不需要专用的I/O指令，而使用访存指令直接访问外部设备。

六、其他指令

■ 除了上述几种类型的指令外，还有一些实现其他控制功能的指令：

- ☐ 停机指令
- ☐ 等待指令
- ☐ 空操作指令
- ☐ 开中断指令
- ☐ 关中断指令
- ☐ 特权指令
  - ✓ 特权指令主要面向操作系统，用于对系统资源的分配和管理，具有特殊的权限，因而不直接提供给用户使用。
  - ✓ 在多任务、多用户的计算机系统中，必须要有特权指令的支持。

■ 在多处理器系统中，还包含有专用的多处理机指令。

七、例题

7.1 例题一

【2011年 题17】某机器有一个标志寄存器，其中有进位/借位标志CF、零标志ZF、符号标志SF和溢出标志OF，条件转移指令bgt（无符号整数比较大小时转移）的转移条件是（C）。

A. $CF+OF=1$ 排除　B. $\overline{SF}+ZF=1$ 排除　C. $\overline{CF+ZF}=1$　D. $\overline{CF+SF}=1$ 排除

解析

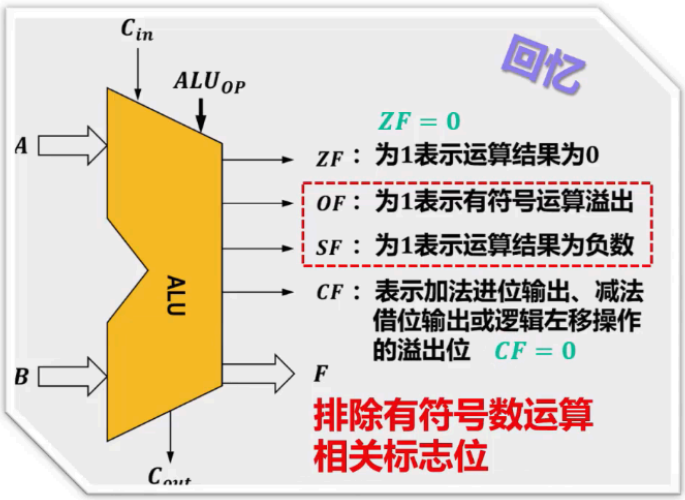
提示

```
mov  r0, #10   ; 将数值 10 传送到寄存器r0
mov  r1, #5    ; 将数值 5传送到寄存器r1

cmp  r0, r1    ; 比较 (r0) 和 (r1)   执行(r0) - (r1)，运算结果影响标志位
bgt  greater   ; 如果 (r0) > (r1)，则跳转到标签 greater

; 继续执行其他指令...

greater:
; 这里是跳转目标
```



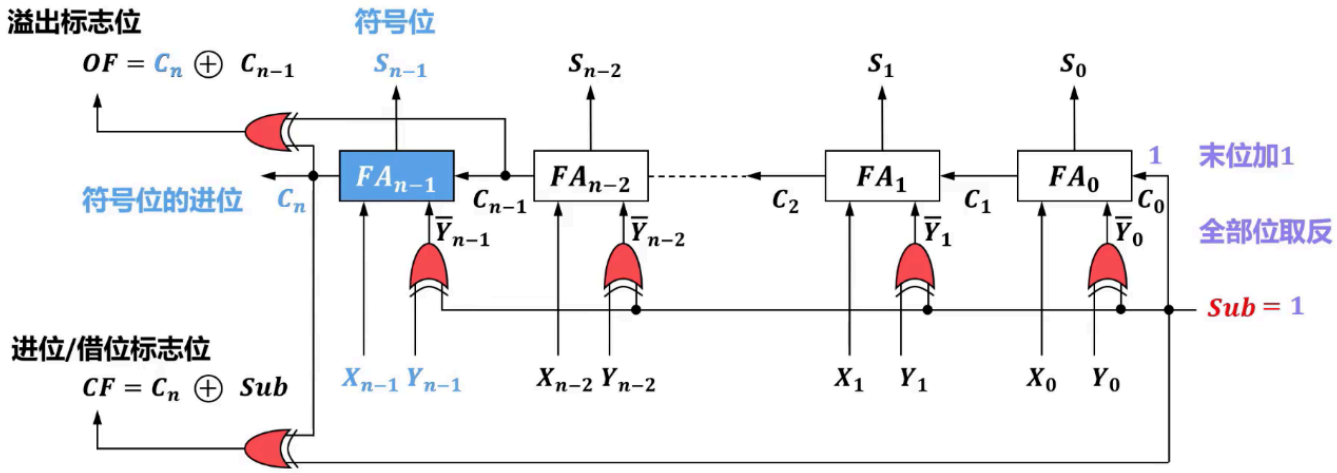
7.2 例题二

【2018年 题19】减法指令“$sub\ R1,\ R2,\ R3$”的功能为“$(R1)-(R2)\rightarrow R3$”，该指令执行后将生成进位/借位标志CF和溢出标志OF。若$(R1)=FFFF\ FFFFH$，$(R2)=FFFF\ FFF0H$，则该减法指令执行后，CF与OF分别为（A）。

A. $CF=0,\ OF=0$　　B. $CF=1,\ OF=0$

C. $CF=0,\ OF=1$　　D. $CF=1,\ OF=1$

基于串行进位加法器讲解



==**CF在无符号和有符号运算中的生态位详解**==

1. 前提：同一电路，语义由软件解释决定

加法器不区分操作数的解释方式，标志位的"有效性"取决于软件对结果的数制解读

2. 数学基础：C_n 的唯一含义

n 位加法器执行模 2^n 加法，真和满足：

$$\text{真和}=C_n\cdot 2^n+S$$

即 C_n **记录"运算结果是否越过模** 2^n **的边界（**$2^n\equiv 0$ **处）"**，无其他信息

3. 无符号数：CF 是完备的越界检测器

无符号值域 $[0,2^n)$，两端均紧贴模边界，故"越界 ⇔ 跨边界 ⇔ C_n 可检测"

加法（Sub=0）：真和 $\geq 2^n \iff C_n=1$，且 $\{CF,S\}$ 拼接即为真值，数据无丢失

```
9 + 8 = 17 > 15:   1001 + 1000 = 1 0001, C₃=1 → CF=1 ✓
```

减法（Sub=1）：电路计算 $X+2^n-Y$，等价于机器预借 2^n 后判断能否归还：

$$C_n=1 \iff X\geq Y\ (\text{无借位}),\qquad C_n=0 \iff X<Y\ (\text{有借位})$$

统一公式 $CF=C_n\oplus Sub$ 将两个方向的检测极性对齐：

运算	越界方向	C_n	CF	语义
加法	真和 $>$ 2^n-1	1	1	上界越界
减法	真差 <0	0	1	下界越界
结论：$CF=1 \iff$ 无符号结果落在 $[0,2^n)$ 之外，与运算方向无关				
#### 4. 有符号数：CF 信息量为零				
补码恒等式 $[x]_{补}+[y]_{补} \equiv x+y \pmod{2^n}$ 表明低 n 位结果**恒为正确真值的模余数。**运算错误仅发生在真值超出 $[-2^{n-1},\ 2^{n-1}-1]$ 时，其电路特征为：				
$$\text{进入符号位的进位} \neq \text{移出符号位的进位} \iff OF=C_n \oplus C_{n-1}=1$$				
C_n 单独对此无诊断能力。4 位最小算例集：				

运算	真值	存储	正确	C_3	CF	OF	说明
$-1+1$	0	0000	✓	1	**1**	0	CF 假报警（模回绕属正常）
$5-6$	-1	1111	✓	0	**1**	0	CF 报告的是无符号命题"$5<6$"
$5+6$	11	1011	✗	0	**0**	1	CF 漏报
$-8+(-8)$	-16	0000	✗	1	1	1	CF 无法区分对错
CF 与结果正确性的四种组合均出现 → **CF 对有符号运算正确性的互信息为零。**原因在于语义错位：有符号值域 $[-2^{n-1},2^{n-1}-1]$ **横跨**模边界（-1 与 0 之间），跨界是补码机制的常态（如 $-1+1$）；而真正的危险边界 $\pm 2^{n-1}$ 的电路签名是 $C_n \neq C_{n-1}$，属 OF 的职责							
#### 5. 对称性：OF 在无符号场景同样失效							

`12 + 12`（`1100 + 1100 = 1 1000`，C_3=1，C_2=1）

无符号：`24 > 15`，溢出。CF=1 ✓ 报警；OF=1⊕1=0 ✗ 漏报

有符号：`(-4)+(-4)=-8`，界内。`1000=-8` ✓ 正确，OF=0 ✓

数据解读	有效检测器	对方标志的表现
无符号	**CF**、ZF	OF 时漏报时误报（12+12 漏报、3+5 误报）
有符号	**OF**、SF、ZF	CF 时漏报时误报（5+6 漏报、−1+1 误报）
#### 6. CF 的延伸职责（均限于无符号/位串场景）		
- **多精度运算**：ADC/SBB 将 CF 作为字间进位/借位传递，实现超字长运算；		
- **带标志移位**：RCL/RCR 经 CF 中转移出位；		
- **显式控制**：STC/CLC/CMC 直接操作 CF		
#### 结论		
C_n 只回答一个问题：**结果是否越过模 2^n 边界。**无符号值域 $[0, 2^n)$ 两端贴合该边界，故 $CF = C_n \oplus Sub$ 构成加、减法通用的完备越界检测器；有符号值域横跨边界，越界为补码常态，正确性检测需依赖符号位进位不一致信号（OF）。CF 在有符号场景并非失效，而是**其所回答的命题（无符号够不够减、是否超过 2^n）在有符号语义下不构成正确性判据**		